

COMPILER DESIGN

LL(1) Violation Analysis Program

Introduction

This C program is designed to analyze whether a grammar violates the **LL(1) parsing condition** using the **FIRST** and **FOLLOW** sets of non-terminals.

In compiler design, an **LL(1) parser** is a top-down parser that reads the input from **Left to right** and produces a **Leftmost derivation using 1 lookahead symbol**.

For a grammar to be LL(1), the parser should be able to decide which production rule to use by looking at only one input symbol.

One important condition is:

- The **intersection of FIRST and FOLLOW sets should be empty** for nullable productions.
- If there is overlap between FIRST and FOLLOW sets, it may create ambiguity during parsing.
- This program detects such overlaps.

Objective of the Program

The main objective of this program is:

- To accept FIRST and FOLLOW sets from the user.
- To compare the characters in both sets.
- To calculate overlap count.
- To determine whether LL(1) condition is violated.
- To display the analysis clearly.

Header Files Used

```
#include <stdio.h>
#include <string.h>
```

Explanation

```
#include <stdio.h>
```

This header file is used for:

- printf()

- scanf()

These functions help in input and output operations.

#include <string.h>

This header file is used for string operations such as:

- Handling character arrays
- Detecting string termination using '\0'

Macro Definition

```
#define MAX 10
```

Explanation

- Defines maximum size for arrays.
- Allows storing up to 10 non-terminals and strings.

This improves readability and avoids magic numbers in the code.

Function Used

```
int intersectionCount(char first[], char follow[])
```

Purpose of the Function

This function calculates how many characters are common between:

- FIRST set
- FOLLOW set

If overlap exists, it may indicate LL(1) violation.

Function Variables

```
int count = 0;
```

- Stores number of overlapping symbols.

Outer Loop

```
for (int i = 0; first[i] != '\0'; i++)
```

Explanation

- Traverses every character in FIRST set.
- Continues until string end ('\0').

Example:

If FIRST = "ab"

Loop checks:

- a
- b

Inner Loop

```
for (int j = 0; follow[j] != '\0'; j++)
```

Explanation

- Traverses every character in FOLLOW set.

Example:

If FOLLOW = "bc"

Loop checks:

- b
- c

Comparison Logic

```
if (first[i] == follow[j])  
{  
    count++;  
}
```

Explanation

- Compares characters from both sets.
- If same character exists in both:
 - overlap count increases.

Example:

FIRST = ab

FOLLOW = bc

Common character:

- b

Therefore:

Overlap Count = 1

Returning Result

```
return count;
```

The function returns total overlapping symbols.

Main Function

```
int main()
```

Execution starts from here.

Variable Declarations

```
int n;  
char nonTerminals[MAX];  
char FIRST[MAX][MAX];  
char FOLLOW[MAX][MAX];
```

Explanation

n

Stores number of non-terminals.

nonTerminals[MAX]

Stores non-terminal symbols.

Example:

A

B

C

FIRST[MAX][MAX]

2D array storing FIRST sets.

Example:

FIRST(A) = ab

FIRST(B) = d

FOLLOW[MAX][MAX]

2D array storing FOLLOW sets.

Example:

$\text{FOLLOW}(A) = bc$

$\text{FOLLOW}(B) = \$$

Taking Number of Non-Terminals

```
printf("Enter number of non-terminals: ");
```

```
scanf("%d", &n);
```

Explanation

User enters total number of grammar variables.

Example:

2

Input Loop

```
for (int i = 0; i < n; i++)
```

This loop accepts details for each non-terminal.

Reading Non-Terminal

```
scanf(" %c", &nonTerminals[i]);
```

Explanation

Reads single character non-terminal.

Example:

A

Reading FIRST Set

```
scanf("%s", FIRST[i]);
```

Explanation

Reads FIRST set as string.

Example:

ab

Means:

$\text{FIRST}(A) = \{a, b\}$

Reading FOLLOW Set

```
scanf("%s", FOLLOW[i]);
```

Explanation

Reads FOLLOW set.

Example:

bc

Means:

$\text{FOLLOW}(A) = \{b, c\}$

Analysis Section

```
printf("\n--- LL(1) Violation Analysis ---\n");
```

Displays heading for results.

Calling Function

```
int overlap = intersectionCount(FIRST[i], FOLLOW[i]);
```

Explanation

- Sends FIRST and FOLLOW sets to function.
- Receives overlap count.

Displaying Results

```
printf("\nNon-terminal: %c", nonTerminals[i]);  
printf("\nFIRST: %s", FIRST[i]);  
printf("\nFOLLOW: %s", FOLLOW[i]);  
printf("\nOverlap Count: %d", overlap);
```

Displays:

- Non-terminal
- FIRST set
- FOLLOW set
- Common symbol count

LL(1) Violation Check

```
if (overlap > 0)
```

Logic

If overlap exists:

- Grammar may violate LL(1) property.

Violation Output

```
printf("\nFlag: Is_LL1_Violated = TRUE\n");
```

Displayed when overlap exists.

Non-Violation Output

```
printf("\nFlag: Is_LL1_Violated = FALSE\n");
```

Displayed when no overlap exists.

Sample Input

Enter number of non-terminals: 2

Enter Non-terminal: A

Enter FIRST set (no spaces): ab

Enter FOLLOW set (no spaces): bc

Enter Non-terminal: B

Enter FIRST set (no spaces): D

Enter FOLLOW set (no spaces): \$

Step-by-Step Execution

For Non-terminal A

FIRST(A)

ab

FOLLOW(A)

bc

Comparison:

FIRST FOLLOW Match

a	b	No
---	---	----

a	c	No
---	---	----

b	b	Yes
---	---	-----

b	c	No
---	---	----

Overlap Count = 1

Result:

Is_LL1_Violated = TRUE

Because:

- Symbol b exists in both sets.

For Non-terminal B

FIRST(B)

D

FOLLOW(B)

\$

No common symbols.

Overlap Count = 0

Result:

Is_LL1_Violated = FALSE

Sample Output

--- LL(1) Violation Analysis ---

Non-terminal: A

FIRST: ab

FOLLOW: bc

Overlap Count: 1

Flag: Is_LL1_Violated = TRUE

Non-terminal: B

FIRST: D

FOLLOW: \$

Overlap Count: 0

Flag: Is_LL1_Violated = FALSE

Time Complexity Analysis

The nested loops compare every character of FIRST with every character of FOLLOW.

If:

- FIRST size = m
- FOLLOW size = n

Complexity:

$O(m \times n)$

Simple Implementation

Easy to understand and execute.

Useful for Compiler Design

Helps analyze LL(1) grammar conditions.

Detects Parsing Conflicts

Identifies overlapping symbols causing ambiguity.

Beginner Friendly

Uses basic:

- loops
- arrays
- strings
- functions

Good for students learning compiler construction.

Limitations

Handles Only Character-Based Sets

Cannot process:

- multi-character terminals
- complex grammar structures

Manual Input Required

User must manually provide FIRST and FOLLOW sets.

Basic LL(1) Check Only

Real LL(1) parsing involves:

- production rules
- nullable productions
- parsing table construction

This program only checks overlap condition.

Conclusion

This program demonstrates a simple method for detecting LL(1) grammar violations using FIRST and FOLLOW set intersection analysis.

By comparing symbols present in both sets, the program identifies possible parsing conflicts that may occur in predictive parsing. It provides a practical understanding of LL(1) concepts used in compiler design and syntax analysis.

The implementation is efficient for small examples and helps students understand:

- FIRST sets
- FOLLOW sets
- LL(1) parsing rules
- grammar ambiguity detection

Overall, the program serves as a useful educational tool for learning predictive parser validation in compiler construction.